

# Automatisierung der Zuordnung von Kundenherausforderungen zu Intershop-Kundenreferenzen

Prototyp · April 2026

Onur Yazicioglu

## 1. Ziel der Lösung

---

Ziel war es, ein System zu entwickeln, das Vertriebsmitarbeiter und Kunden dabei unterstützt, passende Intershop-Kundenreferenzen für konkrete Herausforderungen zu finden. Statt manueller Suche gibt der Nutzer eine Herausforderung ein, zum Beispiel „Wir möchten in 20 Länder verkaufen“ und das System liefert automatisch die relevantesten Referenzen zurück.

Darüber hinaus bietet das System weitere Funktionen, die den Vertriebsprozess unterstützen: Der Nutzer kann alle bisher gespeicherten Herausforderungen einsehen, eine visuelle Übersicht abrufen, die zeigt welche Herausforderungen wie viele Referenzen abdecken, sowie für eine konkrete Herausforderung automatisch eine Produktempfehlung erhalten. Diese Empfehlung wird von einem Large Language Model generiert, das die Herausforderung gegen die verfügbaren Intershop-Produkte abgleicht und die passendsten Optionen mit Begründung zurückgibt.

## 2. Wie die Lösung funktioniert

---

### 2.1 Datenbeschaffung – Web Scraper

Da die Intershop-Seiten keine strukturierte Datenschnittstelle anbieten, wurden zwei Scraper entwickelt, die jeweils unterschiedliche Inhalte automatisch auslesen und lokal speichern.

Der erste Scraper liest die Intershop-Referenzseite aus und speichert die Daten für jedes Unternehmen in eine eigene JSON-Datei. Jede Datei enthält eine kurze Beschreibung des Kundennutzens sowie Metadaten wie Unternehmensname, Logo-URL und, falls vorhanden, einen Link zur Detailseite oder zur Website des Kunden. Dieses Format ist direkt kompatibel mit modernen KI-Frameworks und kann ohne Umwandlung in eine Vektordatenbank geladen werden.

Der zweite Scraper durchläuft mehrere Intershop-Lösungsseiten; darunter B2B, B2C, B2X, Manufacturing, Wholesale und Retail; und extrahiert daraus Überschriften, Beschreibungen und Feature-Listen. Die gesammelten Inhalte werden in einer einzigen Textdatei (products.txt) gespeichert, gegliedert nach Quell-URL. Diese Datei dient später als Wissensbasis für die Produktempfehlung: Sie wird zusammen mit der Herausforderung des Nutzers an das Language Model übergeben, damit dieses eine fundierte und produktspezifische Empfehlung generieren kann.

## 2.2 Vektordatenbank – Semantische Suche

Die gespeicherten Referenztexte werden in numerische Vektoren umgewandelt (Embeddings) und in einer lokalen Vektordatenbank gespeichert. Beim Suchen wird die Nutzeranfrage auf dieselbe Weise kodiert und mit den gespeicherten Referenzen verglichen, nicht durch Stichwortsuche, sondern durch semantische Ähnlichkeit. Das bedeutet: „Wir haben sehr viele Produkte“ findet auch Referenzen, die von „großem Produktkatalog“ oder „80.000 Artikeln“ sprechen, ohne dass diese Worte explizit im Suchbegriff vorkommen.

## 2.3 Herausforderungen – Deduplizierung und Tagging

Neue Herausforderungen werden vor der Speicherung auf Ähnlichkeit mit bereits vorhandenen geprüft. Liegt die semantische Ähnlichkeit über einem definierten Schwellenwert, wird die neue Herausforderung abgelehnt und der Nutzer auf die bestehende verwiesen. So bleibt die Liste der Herausforderungen sauber und ohne Dopplungen.

Wird eine Herausforderung akzeptiert, wird sie automatisch mit allen Referenzen abgeglichen, die inhaltlich dazu passen. Diese Referenzen werden entsprechend markiert (getaggt), sodass spätere Suchanfragen nur Ergebnisse liefern, die nachweislich zur jeweiligen Herausforderung passen.

## 2.4 Gefiltertes Suchergebnis

Bei einer Suchanfrage werden zunächst bis zu 15 Kandidaten aus der Vektordatenbank abgerufen. Danach wird jeder Kandidat anhand der gespeicherten Metadaten gefiltert: Nur Referenzen, die beim Tagging-Schritt als relevant eingestuft wurden, gelangen ins Ergebnis. Von diesen werden maximal 5 an den Nutzer zurückgegeben, sortiert nach semantischer Nähe.

## 2.5 Produktempfehlung

Über einen separaten Endpunkt kann der Nutzer zu einer bekannten Herausforderung eine Produktempfehlung anfordern. Das System sendet dabei die Herausforderung zusammen mit den verfügbaren Intershop-Produktbeschreibungen an ein Large Language Model (Azure AI Foundry). Das Modell analysiert den Kontext und gibt eine begründete Empfehlung zurück, welche Intershop-Produkte am besten zur Situation des Kunden passen.

## 2.6 Visualisierung

Ein Visualisierungsendpunkt zeigt im Terminal an, wie viele Referenzen jeder gespeicherten Herausforderung zugeordnet sind. Dies gibt einen schnellen Überblick, welche Themen im Referenzportfolio besonders gut abgedeckt sind.

# 3. Systemarchitektur

---

Das System ist modular aufgebaut und besteht aus folgenden Komponenten:

- **Scrapers:** Extraktion der Kunden- und Produktdaten von der Intershop-Website
- **Embedding-Modul:** Zentrale Komponente zur Textvektorisierung, einmalig geladen und wiederverwendet
- **Challenge-Modul:** Verwaltung, Deduplizierung und Tagging von Herausforderungen
- **RAG-Modul:** Aufbau und Abfrage der Vektordatenbank inkl. Metadatenfilterung
- **API-Server:** FastAPI-Anwendung mit 6 Endpunkten

- **CLI-Client:** Kommandozeilenwerkzeug für alle Endpunkte

## 4. Verwendete Technologien

---

### KI-Assistenten (Vibe Coding)

Für dieses Projekt habe ich KI-gestützte Coding-Assistenten eingesetzt, sowohl für das initiale Brainstorming als auch für die Implementierung und Dokumentation. Konkret habe ich mit Claude Sonnet 4.6 und GPT-5.3 Instant gearbeitet.

### Python

Projektsprache. Gewählt wegen der breiten Unterstützung im KI- und Datenbereich sowie der Verfügbarkeit aller benötigten Bibliotheken als stabile, gut dokumentierte Pakete.

### FastAPI

Framework für den REST-API-Server. FastAPI ermöglicht die schnelle Entwicklung robuster APIs mit automatischer Validierung und interaktiver Dokumentation. Es ist asynchron-fähig und eignet sich gut für KI-gestützte Backends.

### sentence-transformers / all-MiniLM-L6-v2

Embedding-Modell zur Umwandlung von Texten in semantische Vektoren. Das Modell läuft vollständig lokal ohne API-Schlüssel, ist schnell und für semantische Ähnlichkeitssuche optimiert. Dasselbe Modell wird für Deduplizierung, Tagging und Vektorsuche verwendet, so bleibt der Vektorraum konsistent.

Ich habe mich bewusst für ein kleines, lokales neuronales Netzwerkmodell entschieden, das vollständig offline betrieben werden kann, um zu zeigen, dass ich in der Lage bin, KI-Modelle direkt einzubinden und ohne Abhängigkeit von externen APIs produktiv einzusetzen.

### ChromaDB

Lokale Vektordatenbank zur Speicherung und Abfrage der Referenz-Embeddings. ChromaDB persistiert Daten auf der Festplatte und bietet eine direkte Integration mit LangChain. Es ist keine separate Datenbankinstallation erforderlich.

### LangChain

Framework zur Orchestrierung der RAG-Pipeline. LangChain stellt die Document-Abstraktion bereit, über die JSON-Dateien mit Inhalt und Metadaten strukturiert geladen und an ChromaDB übergeben werden.

### BeautifulSoup / Requests

Bibliotheken für das Web-Scraping. Requests übernimmt den HTTP-Abruf der Intershop-Seite, BeautifulSoup das strukturierte Parsen des HTML. Die Kombination ist leichtgewichtig und gut geeignet für server-seitig gerenderte Seiten.

### scikit-learn

Wird für die Kosinus-Ähnlichkeitsberechnung bei der Deduplizierung von Herausforderungen eingesetzt. Die Bibliothek ist numerisch stabil und weit verbreitet.

## Azure AI Foundry (Azure OpenAI)

LLM-Dienst für die Produktempfehlung. Herausforderung und Produktbeschreibungen werden an ein GPT-Modell übergeben, das eine kontextualisierte Empfehlung zurückgibt. Azure AI Foundry wurde gewählt, da es in vielen Unternehmensumgebungen bereits lizenziert und DSGVO-konform verfügbar ist.

Ich habe mich bewusst für eine Online-KI-Plattform entschieden, um zu zeigen, dass ich cloud-basierte KI- und Machine-Learning-Dienste gezielt einsetzen kann.

## 5. API-Endpunkte im Überblick

---

- **POST /challenge/new:** Neue Herausforderung einreichen (inkl. Deduplizierung, Tagging, automatische Suche)
- **GET /search:** Referenzsuche für eine bekannte Herausforderung
- **GET /visualization:** Übersicht: Herausforderung → Anzahl getaggtter Referenzen
- **GET /product-recommendation:** Produktempfehlung via Azure LLM
- **GET /challenges:** Liste aller gespeicherten Herausforderungen
- **GET /endpoints:** Übersicht aller verfügbaren Endpunkte

## 6. Erweiterbarkeit

---

Das System ist bewusst modular gehalten. Einzelne Komponenten können unabhängig ausgetauscht oder erweitert werden:

- **Frontend:** Aktuell erfolgt die Bedienung über ein CLI-Tool. Da das System auf einer REST-API basiert, lässt sich jederzeit ein Web-Frontend ergänzen
- **Embedding-Modell:** Austausch durch fortgeschrittene OpenAI- oder Azure-Embeddings durch Änderung einer einzigen Zeile
- **Detailseiten der Referenzen:** Die in den JSON-Dateien gespeicherten „Read more“-Links wurden bisher nicht genutzt. Die verlinkten Detailseiten könnten zusätzlich gescrapt und in die Suche einbezogen werden. Da diese Texte jedoch deutlich länger und strukturell anders sind als die kurzen Referenzbeschreibungen, würde es Sinn machen, zwei separate Vektordatenbanken aufzubauen.
- **Zero-Shot-Klassifikation:** Statt Embeddings für die Zuordnung von Herausforderungen zu Referenzen zu verwenden, könnten Zero-Shot-Klassifikatoren eingesetzt werden. Dies würde eine präzisere Kategorisierung ermöglichen, allerdings auf Kosten der Flexibilität, das Hinzufügen neuer Herausforderungen wäre damit aufwändiger.
- **Unit Tests:** Das Hinzufügen von Unit Tests für die einzelnen Module, insbesondere für die Deduplizierungslogik, das Tagging und die RAG-Pipeline, würde die Zuverlässigkeit des Systems erhöhen und spätere Erweiterungen absichern.